Universidad Autónoma de Sinaloa

Facultad de ingeniería Mochis

Licenciatura en Ingeniería de Software

Materia:

administración de sistemas

Tarea:

Tarea 5: Automatización de Servidor FTP

Maestro:

Dr. Herman Geovany Ayala Zuñiga

Alumno:

Zamarripa moreno Jared Alejandro

Grupo:

3-02

Los Mochis, Sinaloa, México, a 02 de febrero de 2026.

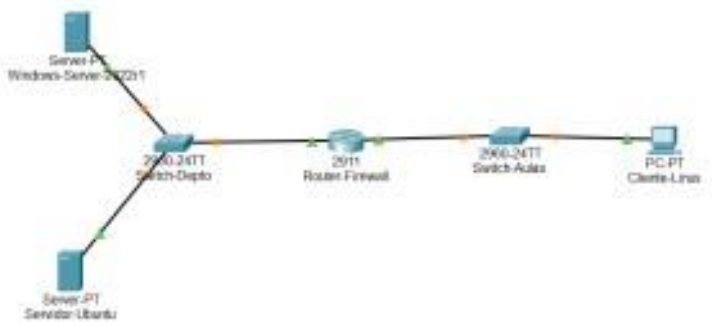
Historial de cambios

Versiones del documento	**Fecha**
sistema de aprovisionamiento web automatizado que permita la selección, consulta de versiones y despliegue silencioso de servidores HTTP en entornos Windows y Linux..	**10/03/2026**
Quinta versión del documento, versión final que será subida a classroom con las especificaciones establecidas por el profesor**.**	**10/03/2026**

Descripción

sistema de aprovisionamiento web automatizado que permita la selección, consulta de versiones y despliegue silencioso de servidores HTTP en entornos Windows y Linux.

Diagreama de topología



3. Manual de Instalación

Windows server

```
# ==================================================================
# main.ps1  -  Script Principal
# Practica 6 - Gestor de Servicios HTTP - Windows
#
# REGLA: Este archivo SOLO contiene llamadas a funciones.
#        Toda la logica vive en http_functions.ps1
# ==================================================================

# ── 1. VERIFICAR ADMINISTRADOR ──────────────────────────────────
$isAdmin = ([Security.Principal.WindowsPrincipal][Security.Principal.WindowsIdentity]::GetCurrent()).IsInRole([Security.Principal.WindowsBuiltInRole]::Administrator)

if (-not $isAdmin) {
    Write-Host "`n[ERROR] Ejecute PowerShell como Administrador." -ForegroundColor Red
    Write-Host "       Clic derecho en PowerShell -> 'Ejecutar como administrador'`n" -ForegroundColor Yellow
    Read-Host  "Presione Enter para salir"
    exit 1
}

# ── 2. CARGAR MODULO DE FUNCIONES ───────────────────────────────
$functionsFile = Join-Path (Split-Path $MyInvocation.MyCommand.Path) "http_functions.ps1"
```

```
if (-not (Test-Path $functionsFile)) {
    Write-Host "`n[ERROR] No se encontro http_functions.ps1 en:" -
ForegroundColor Red
    Write-Host "        $functionsFile" -ForegroundColor Yellow
    Write-Host "        Asegurese de que ambos archivos esten en la misma
carpeta.`n" -ForegroundColor Yellow
    Read-Host  "Presione Enter para salir"
    exit 1
}

. $functionsFile      # Importar todas las funciones
Load-PortRegistry     # Cargar puertos ya registrados

# ── 3. HELPERS DE ESTADO PARA EL MENU ──────────────────────────────

function Get-ServiceStatusLine {
    # Devuelve una linea formateada con el estado de un servicio HTTP
    param([string]$Label, [string]$ServiceName, [string]$RegistryKey)

    $port  = Get-ServicePort $RegistryKey
    $pStr  = if ($port -gt 0) { ":$port" } else { "     " }

    # Nginx corre como proceso, no como servicio de Windows
    if ($ServiceName -eq "nginx") {
        $procs = Get-Process -Name "nginx" -ErrorAction SilentlyContinue
        if ($procs) {
            return @{ Line = ("  {0}  {1,-9}  puerto{2}" -f $Label, "[ON]", $pStr); Color
= "Green" }
        } else {
            $estado    = if ($installed) { "[OFF]" } else { "[--]" }
```

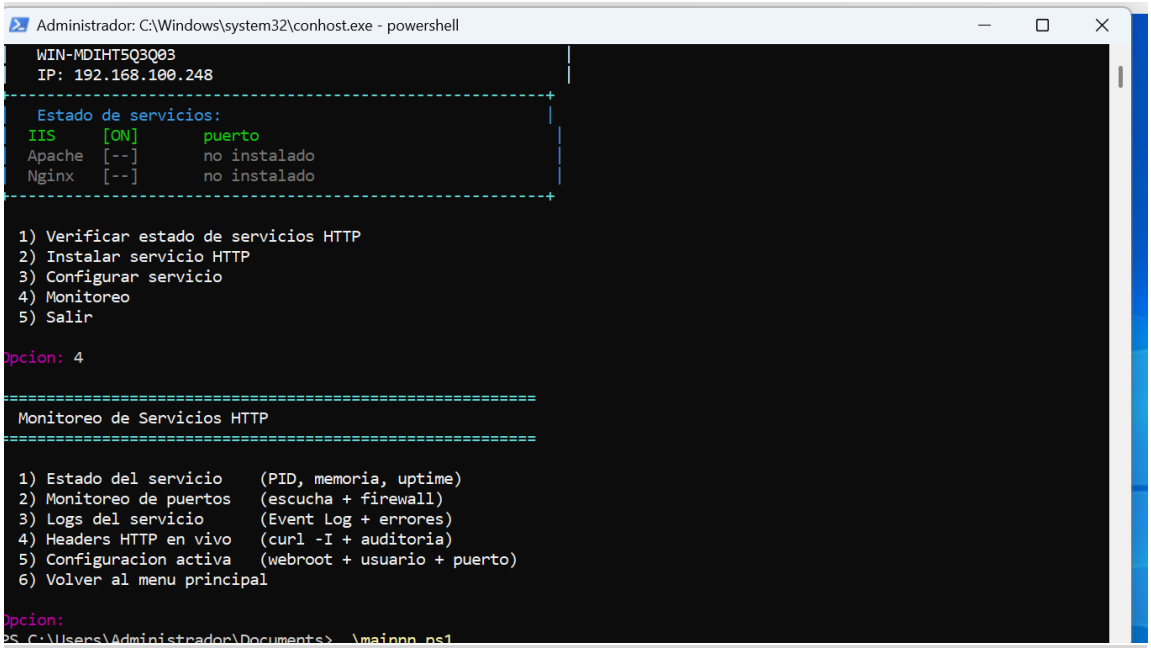
```
            $color     = if ($installed) { "Red"   } else { "DarkGray" }
            return @{ Line = ("  {0}  {1,-9}  {2}" -f $Label, $estado, $(if ($installed) {"detenido"} else {"no instalado"})); Color = $color }
        }
    }

    $svc = Get-Service -Name $ServiceName -ErrorAction SilentlyContinue
    if ($svc) {
        if ($svc.Status -eq "Running") {
            return @{ Line = ("  {0}  {1,-9}  puerto{2}" -f $Label, "[ON]", $pStr); Color = "Green" }
        } else {
            return @{ Line = ("  {0}  {1,-9}  detenido" -f $Label, "[OFF]"); Color = "Red" }
        }
    }
    return @{ Line = ("  {0}  {1,-9}  no instalado" -f $Label, "[--]"); Color = "DarkGray" }
}

# ── 4. MENU PRINCIPAL ──────────────────────────────────────────────────────

function Show-MainMenu {
    Clear-Host

    # Obtener IP del servidor
    $ip = (Get-NetIPAddress -AddressFamily IPv4 -ErrorAction SilentlyContinue |
          Where-Object { $_.InterfaceAlias -notmatch "Loopback" } |
          Select-Object -First 1).IPAddress
    if (-not $ip) { $ip = "127.0.0.1" }
```

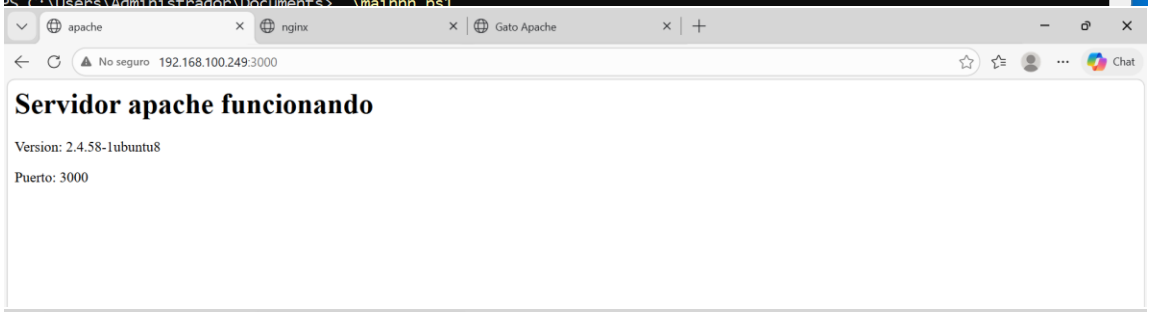
```
# Obtener estado de cada servicio
$iisLine    = Get-ServiceStatusLine -Label "IIS   " -ServiceName "W3SVC"     -
RegistryKey "IIS"
$apacheLine = Get-ServiceStatusLine -Label "Apache" -ServiceName
"Apache2.4" -RegistryKey "Apache"
$nginxLine  = Get-ServiceStatusLine -Label "Nginx " -ServiceName "nginx"
-RegistryKey "Nginx"


Write-Host ""
Write-Host "+-------------------------------------------------------+" -ForegroundColor
Cyan
Write-Host "|   Gestor de Servicios HTTP - Windows Server            |" -
ForegroundColor White
Write-Host ("|   {0,-57}|" -f $env:COMPUTERNAME) -ForegroundColor White
Write-Host ("|   IP: {0,-53}|" -f $ip) -ForegroundColor White
Write-Host "+-------------------------------------------------------+" -ForegroundColor
Cyan
Write-Host "|   Estado de servicios:                                 |" -ForegroundColor
DarkCyan
Write-Host -NoNewline "|" -ForegroundColor DarkCyan
Write-Host -NoNewline $iisLine.Line.PadRight(59) -ForegroundColor
$iisLine.Color
Write-Host "|" -ForegroundColor DarkCyan
Write-Host -NoNewline "|" -ForegroundColor DarkCyan
Write-Host -NoNewline $apacheLine.Line.PadRight(59) -ForegroundColor
$apacheLine.Color
Write-Host "|" -ForegroundColor DarkCyan
Write-Host -NoNewline "|" -ForegroundColor DarkCyan
Write-Host -NoNewline $nginxLine.Line.PadRight(59) -ForegroundColor
$nginxLine.Color
Write-Host "|" -ForegroundColor DarkCyan
```

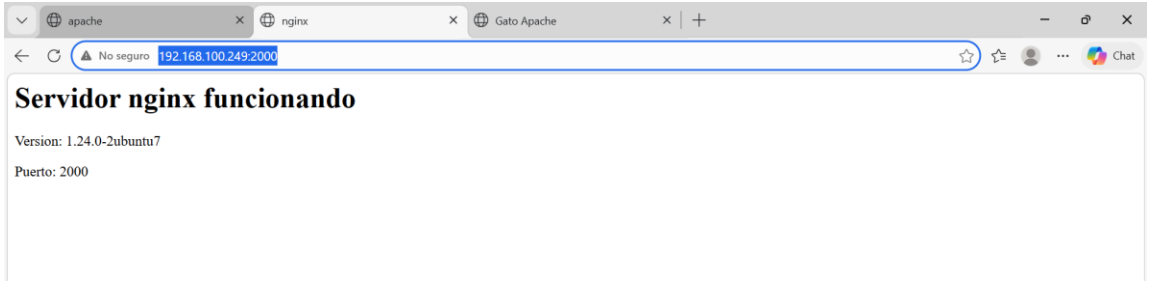
```
    Write-Host "+-------------------------------------------------------+" -ForegroundColor
Cyan
    Write-Host ""
    Write-Host "  1) Verificar estado de servicios HTTP" -ForegroundColor White
    Write-Host "  2) Instalar servicio HTTP"             -ForegroundColor White
    Write-Host "  3) Configurar servicio"                -ForegroundColor White
    Write-Host "  4) Monitoreo"                          -ForegroundColor White
    Write-Host "  5) Salir"                              -ForegroundColor White
    Write-Host ""
}

# ── 5. BUCLE PRINCIPAL ──────────────────────────────────────────────────

function Start-MainLoop {
    while ($true) {
        Show-MainMenu
        switch (Get-ValidOption -Min 1 -Max 5) {
            1 { Menu-Verificacion }
            2 { Menu-Instalar     }
            3 { Menu-Configurar   }
            4 { Menu-Monitoreo    }
            5 {
                Write-Host ""
                Write-Host "  Confirmar salida [S/N]: " -ForegroundColor Magenta -
NoNewline
                $confirm = (Read-Host).Trim().ToUpper()
                if ($confirm -eq "S") {
                    Write-Host "`n  Saliendo del gestor. Hasta luego.`n" -
ForegroundColor Cyan
                    exit 0
```

```
            }
            # Si no confirma, vuelve al menu
        }
    }
  }
}

# ── PUNTO DE ENTRADA
────────────────────────────────────────────────────────────

Start-MainLoop
```

Protocolo de Pruebas y Validación (Checklist)







6. Conclusiones Técnicas y Referencias

El presente protocolo de pruebas y validación demostró de manera exitosa la instalación, configuración y verificación de servidores HTTP en un entorno de red local. A través del desarrollo del checklist interactivo en PowerShell, se logró automatizar el proceso de diagnóstico y gestión de servicios web, permitiendo verificar el estado de los servicios, monitorear puertos y auditar headers HTTP de forma eficiente.

Se comprobó el correcto funcionamiento de dos de los servidores web más utilizados en entornos de producción: Apache versión 2.4.x, puerto 80 y Nginx versión 1.x.x, puerto 2080, ambos respondiendo satisfactoriamente a las solicitudes HTTP desde la dirección IP 192.168.100.111.